MyCodeSchool

Angular — componente, servicii, rute, formulare

Teoria pentru User Management fără server · Angular 22 · Sebastian

1. Componenta = o clasă + un template

O componentă e o bucată de ecran: **clasa** (`.ts`) ține datele și metodele, **template-ul** (`.html`) spune cum arată. **Selectorul** e numele tag-ului prin care o pui în altă pagină.

Creare — din terminal, în folderul proiectului

```
npx ng g c users/user-list --flat --skip-tests --style=none
```

`g c` = *generate component*; `--flat` = fără subfolder; `--skip-tests` și `--style=none` = fără fișierul de teste și fără CSS (stilurile sunt globale, în `styles.css`). Apar două fișiere:

users/user-list.ts

```
import { Component } from '@angular/core';

@Component({
  imports: [],
  selector: 'app-user-list',
  templateUrl: './user-list.html',
})
export class UserList {}
```

users/user-list.html

```
<p>user-list works!</p>
```

Folosirea în altă componentă (ex. `App`)

```
imports: [UserList],          // în .ts
<app-user-list />             <!-- în .html -->
```

Uiți `imports` → eroare: `'app-user-list' is not a known element`.

2. Ce scrii în template

Sintaxă	Ce face	Exemplu
`{{ }}`	afișează o valoare	`{{ user.firstName }}`, `{{ total() }}`
`[prop]="..."`	dă o valoare unei proprietăți / intrări	`[value]="search()"`, `[disabled]="form.invalid"`
`[class.x]="..."`	pune clasa `x` doar când e `true`	`[class.badge-warning]="user.age < 18"`
`(event)="..."`	cheamă o metodă la un eveniment	`(click)="remove(user)"`
`#nume`	referință la un element din pagină	`<input #box (input)="search.set(box.value)">`
`@if / @else`	afișează doar dacă	`@if (user(); as user) { ... } @else { 404 }`
`@for … @empty`	repetă; `@empty` când lista e goală	`@for (user of users(); track user.id) { ... }`

Semnalele se citesc **cu paranteze**: `users()`. Fără ele primești funcția, nu lista.

3. Părinte ↔ copil: `input()` și `output()`

Copilul (stat-card.ts)

```
export class StatCard {
  readonly label = input.required<string>();
  readonly value = input.required<number>();
}
```

Copilul care anunță (confirm-dialog.ts)

```
readonly confirmed = output<void>();
<button
(click)="confirmed.emit()">Sterge</button>
```

Părintele (user-list.html)

```
<app-stat-card label="Minori"
               [value]="minors()" />

<app-confirm-dialog
  [message]="..."
  (confirmed)="remove(user)"
  (cancelled)="toDelete.set(null)" />
```

Datele **coboară** prin `input()` (cu `[ ]`); evenimentele **urcă** prin `output()` (cu `( )`). Copilul nu știe cine e părintele; doar anunță.

4. Serviciul = datele comune tuturor componentelor

```
npx ng g s services/user --type=service --skip-tests
```

services/user.service.ts

```
@Service()
export class UserService {
  private readonly users = signal<User[]>
(SEED_USERS);
  readonly all = this.users.asReadonly();

  remove(id: string): void {
    this.users.update((list) =>
      list.filter((user) => user.id !== id));
  }
}
```

În orice componentă

```
private readonly service = inject(UserService);
protected readonly users = this.service.all;
```

`@Service()`: Angular face **un singur** obiect `UserService` pentru toată aplicația, iar `inject()` ți-l dă (ca un `@Autowired` din Spring). Lista, detaliul și formularul văd aceleași date.

Doar serviciul modifică `users`; componentele citesc `all()`. Când lista se schimbă, tot ce o citește se redesenează singur.

Fără `--type=service`, Angular 22 creează `services/user.ts` cu clasa `User`, care se ciocnește cu interfața `User`.

5. Rute: o adresă = o pagină

app.routes.ts

```
export const routes: Routes = [
  { path: '', redirectTo: 'users', pathMatch:
'full' },
  { path: 'users', component: UserList },
  { path: 'users/new', component: UserForm },
  { path: 'users/:id', component: UserDetail },
];
```

app.config.ts

```
provideRouter(routes,
withComponentInputBinding())
```

În template

```
<router-outlet />   <!-- aici apare pagina -->
<a routerLink="/users/new">Nou</a>
<a [routerLink]="['/users', user.id]">...</a>
```

Parametrul `:id` ajunge în componentă

```
readonly id = input.required<string>();
```

Rutele se verifică **de sus în jos**, prima potrivită câștigă. Din cod: `inject(Router).navigate(['/users'])`.

6. Formulare reactive

În clasă (`imports: [ReactiveFormsModule]`)

```
form = inject(FormBuilder).nonNullable.group({
  email: ['', [Validators.required,
               Validators.email]],
  age: [0, [Validators.min(1)]],
});

this.form.getRawValue();   // { email, age }
this.form.patchValue(user); // completează
```

În template

```
<div [formGroup]="form">
  <input formControlName="email">
  @if (form.controls.email.invalid
       && form.controls.email.touched) {
    <span class="form-error">...</span>
  }
</div>
<button
[disabled]="form.invalid">Salveaza</button>
```

`touched` devine `true` după ce ieși din câmp, de aceea erorile nu apar la deschiderea paginii. Un control `disable()`-uit nu se mai verifică.